

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МО ЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №5**  
**по дисциплине Построение и анализ алгоритмов**

Студент гр. 4343

Роменский М.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

## Задание

### Базовая часть:

#### . Реализация алгоритма Ахо-Корасик для множественного поиска образцов

- **Входные данные:** Текст  $T$  ( $1 \leq |T| \leq 100\,000$ ), количество шаблонов  $n$  ( $1 \leq n \leq 3000$ ) и набор строк-образцов  $P = \{p_1, \dots, p_n\}$  ( $1 \leq |p_i| \leq 75$ ). Используемый алфавит:  $\{A, C, G, T, N\}$ .
- **Выходные данные:** Список пар  $(i, p)$ , отсортированный по возрастанию. Здесь  $i$  обозначает позицию начала вхождения (нумерация с 1), а  $p$  — порядковый номер шаблона (нумерация с 1).

#### 2. Точный поиск одного образца с использованием джокера

- **Входные данные:** Текст  $T$ , шаблон  $P$  ( $1 \leq |P| \leq 40$ ) и символ джокера.
- **Выходные данные:** Отсортированный список позиций, на которых найдены вхождения шаблона.

### Индивидуальное задание (Вариант 1):

Условие модификации: Джокер может соответствовать любому символу алфавита, за исключением одного конкретного запрещённого символа. То есть совпадение происходит со всеми буквами, кроме заданной.

### Требования к программной реализации:

- В структуре автомата обязательно должны присутствовать и применяться как суффиксные, так и конечные (терминальные/сжатые) ссылки.

- Программа должна поддерживать режим отладки, при котором выводятся этапы построения бора и автомата, детальное описание вершин, а также процесс обработки текста.

## **Описание алгоритма**

### **Базовый алгоритм Ахо-Корасик**

Данный метод позволяет обнаружить все вхождения набора строк-образцов в текст за время, линейно зависящее от длины текста и суммарной длины всех образцов. Процесс работы делится на три ключевых этапа:

**1. Построение префиксного дерева (бора):** Все шаблоны последовательно добавляются в дерево. Каждая вершина структуры содержит следующие поля:

- `children`: словарь (хэш-таблица) для хранения переходов по символам;
- `pattern_indices`: список индексов тех шаблонов, которые заканчиваются в данной вершине;
- `suffix_link`: ссылка на вершину, соответствующую наибольшему собственному суффиксу текущей строки, который одновременно является префиксом какого-либо из шаблонов;
- `terminal_link`: ссылка на ближайшую терминальную вершину (конец шаблона) в цепочке суффиксных ссылок.

**2. Формирование автомата (обход в ширину):** Суффиксные и конечные ссылки вычисляются путём BFS, начиная от корня:

- Для непосредственных потомков корня значение `suffix_link` устанавливается равным 0 (корню).
- Для остальных вершин `suffix_link` определяется путём перехода по суффиксной ссылке родителя до тех пор, пока

не будет найден узел с переходом по текущему символу или не достигнут корень.

- Вычисление `terminal_link` представляет собой важную оптимизацию: если вершина, на которую указывает `suffix_link`, сама является терминальной, то `terminal_link` ведёт на неё. В противном случае он наследует значение `terminal_link` от вершины `suffix_link`. Это позволяет находить все совпадения за константное время на каждом шаге, избегая полного подъёма по суффиксным ссылкам.

### **3. Поиск в тексте:** Обработка текста производится посимвольно.

Если прямой переход по текущему символу отсутствует, алгоритм выполняет «откат» по суффиксным ссылкам. При попадании в новую вершину фиксируются все найденные совпадения: сначала проверяются шаблоны в `pattern_indices` текущей вершины, затем осуществляется проход по цепочке `terminal_link` для обнаружения остальных подходящих образцов.

## **2.2. Модификация для Варианта 1 (Джокер с исключением)**

Поскольку классическая версия алгоритма Ахо-Корасик не поддерживает работу с подстановочными знаками напрямую, используется следующий подход:

1. Исходный шаблон разбивается символом-джокером на массив непрерывных подстрок (фрагментов). Для каждого фрагмента сохраняется его начальная позиция внутри исходного шаблона.
2. На основе найденных фрагментов строится стандартный автомат Ахо-Корасик.
3. Автомат сканирует текст и находит все вхождения этих фрагментов.

4. Используется вспомогательный массив счётчиков *occurrences*. Когда фрагмент шаблона обнаруживается в тексте, вычисляется потенциальная стартовая позиция всего шаблона, и соответствующий счётчик увеличивается на единицу.
5. Позиция в тексте признаётся кандидатом на полное совпадение только тогда, когда её счётчик равен общему количеству фрагментов шаблона (то есть все части были найдены на своих расчётных местах).
6. **Финальная проверка (специфика Варианта 1):** Для каждого кандидата выполняется дополнительная верификация символов текста, находящихся на позициях джокеров. Если хотя бы один из этих символов совпадает с запрещённым, данное вхождение отвергается. В противном случае позиция включается в итоговый ответ.

### 2.3. Оценка сложности

#### **Временная сложность:**

- Построение бора:  $O(M)$ , где  $M$  — суммарная длина всех шаблонов.
- Вычисление ссылок (BFS):  $O(M)$  амортизировано. Каждая суффиксная ссылка либо увеличивает глубину текущего состояния, либо уменьшается, но не более чем на единицу за шаг.
- Поиск в тексте:  $O(|T| + K)$ , где  $|T|$  — длина текста, а  $K$  — общее количество найденных вхождений.
- Финальная проверка для Варианта 1:  $O(|T| \cdot W)$ , где  $W$  — число джокеров в шаблоне ( $W \leq 40$ ).
- **Итоговая временная сложность:**  $O(M + |T| + K + |T| \cdot W)$ .

#### **Пространственная сложность:**

- Хранение бора:  $O(M)$  вершин. Применение хэш-таблицы (dict в Python) для переходов children обеспечивает среднее время доступа  $O(1)$  и расход памяти, пропорциональный лишь реально существующим рёбрам, а не размеру всего алфавита  $|\Sigma|$ . Для разреженных деревьев это эффективнее использования массивов фиксированного размера.
- Массивы счётчиков и результатов:  $O(|T|)$ .
- **Итоговая пространственная сложность:**  $O(M + |T|)$ .

### **Тестирование:**

#### **Задача 1: Ахо-Корасик (базовый поиск)**

| Входные данные (T, n, шаблоны) | Выходные данные (i, p)       | Комментарий               |
|--------------------------------|------------------------------|---------------------------|
| NTAG; 3; TAGT, TAG, T          | 2 2; 2 3                     | Базовый тест из Stepik    |
| ACGTAACGT; 3; ACG, GTA, CGT    | 1 1; 2 2; 3 3; 6 1; 7 3      | Пересекающиеся вхождения  |
| AAAA; 1; A                     | 1 1; 2 1; 3 1; 4 1           | Шаблон из одного символа  |
| AC; 1; ACGT                    | (пусто)                      | Шаблон длиннее текста     |
| AGC; 1; TAT                    | (пусто)                      | Шаблон не найден в тексте |
| AAA; 2; A, A                   | 1 1; 1 2; 2 1; 2 2; 3 1; 3 2 | Дубликаты шаблонов        |

#### **Ахо-Корасик с джокером (Вариант 1 — джокер с исключением)**

| Входные данные (T, P, джокер, запрещ.) | Выходные данные (позиции) | Комментарий |
|----------------------------------------|---------------------------|-------------|
|----------------------------------------|---------------------------|-------------|

|                              |         |                                        |
|------------------------------|---------|----------------------------------------|
| ACTANCA; A\$\$A\$;<br>\$; \$ | 1       | Базовый тест из<br>Stepik              |
| ACGAG; A\$A; \$; G           | (пусто) | На месте джокера<br>запрещённый символ |
| ACGAG; A\$A; \$; T           | 1       | На месте джокера<br>допустимый символ  |
| AAAAA; A\$A; \$; X           | 1; 2; 3 | Пересекающиеся<br>вхождения            |
| AAA; \$\$\$; \$; X           | 1       | Шаблон только из<br>джокеров           |
| AC; A\$A\$; \$; \$           | (пусто) | Шаблон длиннее<br>текста               |
| ACGTACGT; G\$G; \$;<br>A     | 1; 5    | Джокер не может<br>быть A              |

### Вывод

В результате выполнения лабораторной работы был разработан алгоритм Ахо-Корасик для решения задачи множественного поиска подстрок в тексте. При реализации особое внимание уделялось соблюдению требований к эффективности: структура автомата включает как суффиксные, так и конечные (терминальные) ссылки. Их корректное применение обеспечивает оптимальную временную сложность этапа сканирования текста  $O(|T| + K)$  и исключает падение производительности при обработке больших объёмов данных.

Для организации переходов между состояниями бора использовались хэш-таблицы (dict). Данное решение гарантирует среднее время доступа  $O(1)$  и позволяет оптимизировать расход

памяти по сравнению с массивами фиксированной длины, сохраняя высокую скорость обработки без избыточного потребления ресурсов.

В рамках индивидуального задания (Вариант 1) базовый алгоритм был адаптирован для работы с джокерами, имеющими ограничение на совпадение. Реализован метод декомпозиции шаблона на непрерывные фрагменты, их поиск посредством классического автомата и последующая верификация позиций подстановочных знаков на отсутствие запрещённого символа. Также был внедрён механизм детального отладочного вывода, позволяющий продемонстрировать полный цикл работы программы: от формирования префиксного дерева до пошаговой обработки входного текста автоматом.